

УЧЕБНИК ДЛЯ ТЕХ, КТО
НИКОГДА НЕ ПРОГРАММИРОВАЛ

Сайт с нуля

Как собрать интернет-магазин
и стать fullstack-разработчиком

60 глав · практика на живом коде

autodoc.tj

Содержание

- 1** Что такое сайт и что мы построим
 - 2** Как работает интернет
 - 3** Инструменты: редактор, терминал, браузер
 - 4** Первый файл на экране
-

Книга пишется по частям. Здесь — главы, готовые на сегодня.

Что такое сайт и что мы построим

ЗАЧЕМ ЭТО

Зачем эта глава

Здравствуйте. Если вы открыли эту книгу, скорее всего, вы уже пробовали подступиться к программированию — и что-то пошло не так. Так бывает почти у всех, и дело обычно не в вас.

Смотрите, что происходит на типичных курсах. Первые две недели вам рассказывают про «переменные», «циклы» и «типы данных». Всё это правда нужно. Но вы сидите и думаете: *а где, собственно, сайт?* Знания вроде появляются, а куда их приложить — непонятно. Это как учиться водить, изучая устройство карбюратора: полезно, но до машины вы так и не дошли.

Поэтому первая глава — не про код вообще. Она про **карту местности**.

Мы разберёмся, из каких частей состоит любой сайт, кто эти части делает и как они между собой связаны. И когда в главе 20 появятся те самые переменные, вы уже будете точно знать, в каком месте магазина они живут и зачем нужны.

Никакого кода набирать пока не надо. Просто читайте — как будто вам показывают дом перед тем, как дать в руки инструменты.

РАЗБИРАЕМСЯ

Начнём с обычного магазина

Забудьте на минуту про компьютеры. Правда, совсем забудьте.

Представьте магазин автозапчастей на вашей улице. Тот самый, куда вы заходили за щётками стеклоочистителя. Что там есть?

Во-первых, торговый зал. Полки, товар, ценники, витрина у входа, касса. Всё разложено так, чтобы покупатель разобрался сам: колодки — сюда, фильтры — туда, акция — на видном месте. Здесь красиво и понятно. Покупатель дальше этой зоны не заходит.

Во-вторых, подсобка. Дверь с табличкой «Посторонним вход воспрещён». Там принимают товар, считают выручку, оформляют накладные, решают, кому дать скидку, а кому нет. Там же лежат деньги. Покупателю туда нельзя — и это не из вредности, а просто потому, что иначе цены на ценниках начнут переписывать все кому не лень.

В-третьих, складской журнал. Толстая тетрадь или программа на компьютере, где записано: какой товар, сколько штук, почём куплено, почём продаём, кто привёз, кому отгрузили. Скучная штука. Но потеряйте её — и магазин превратится в склад коробок, про которые никто ничего не знает.

Всё. Три зоны. Держите эту картинку в голове — сейчас будет главное.

► Сайт устроен ровно так же

Не «похоже». Не «примерно». А один в один:

В МАГАЗИНЕ НА УЛИЦЕ	НА САЙТЕ	КАК ЭТО ЗОВУТ ПРОГРАММИСТЫ
Торговый зал	То, что видно в браузере	Фронтенд (frontend — «передняя часть»)
Подсобка	Программа на сервере	Бэкенд (backend — «задняя часть»)
Складской журнал	База данных	БД (database)

Когда вы заходите на любой сайт — маркетплейс, банк, доставку еды — вы всегда стоите в торговом зале. Всё остальное происходит за дверью, куда вас не пускают.

А человек, который умеет работать **во всех трёх зонах**, называется **fullstack-разработчик**. Дословно — «полный набор». Именно им вы и станете к концу книги.

► Ещё немного про эти три зоны

Чтобы совсем закрепилось, вот та же тройка на других примерах.

Ресторан. Зал со столиками и меню — фронтенд. Кухня, где готовят и считают себестоимость, — бэкенд. Список продуктов на складе и книга заказов — база данных.

Поликлиника. Регистратура и коридор с креслами — фронтенд. Кабинет врача, где принимают решения, — бэкенд. Картотека с медкартами — база данных.

Такси в телефоне. Карта с машинками и кнопка «Заказать» — фронтенд. Программа, которая ищет ближайшего водителя и считает цену, — бэкенд. Хранилище поездок, водителей и оценок — база данных.

Заметили? Везде одно и то же: **красивая витрина, скрытая от глаз логика и место, где всё записано.** Любой сайт, любое приложение, любой сервис.

РАЗБИРАЕМСЯ

Пять инструментов, и всё

Хорошая новость: чтобы построить магазин, нужно не пятьдесят технологий. Нужно **пять**. Давайте познакомимся с каждой — с примерами, чтобы не осталось тумана.

► HTML — скелет

HTML отвечает на вопрос «**что здесь есть?**». Не «как выглядит», а именно «что есть».

Он говорит браузеру: тут заголовок, тут абзац текста, тут картинка, тут кнопка, тут список из трёх пунктов, тут таблица. И всё — на этом его работа заканчивается. Красотой он не занимается принципиально.

Примеры того, что делает HTML:

- на странице появился заголовок «Тормозные колодки»;
- под ним — абзац с описанием;
- слева — фотография товара;
- ниже — список характеристик: производитель, артикул, вес;
- в конце — кнопка «Купить».

Аналогия: скелет человека. Есть череп, рёбра, руки, ноги. На свидание в таком виде не пойдёшь, но сразу понятно: это человек, вот голова, вот конечности.

Ещё точнее — **чертёж дома**. На чертеже нет обоев и мебели, но видно: тут комната, тут окно, тут дверь.

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

Это уже настоящая, рабочая страница. Уродливая до слёз — но рабочая. Браузер её откроет.

► CSS — внешность

CSS отвечает на вопрос «**а как это выглядит?**».

Тот же самый скелет, но теперь у него появились цвет, шрифт, размер, отступы, тени, скруглённые углы и понятное расположение на экране.

Примеры того, что делает CSS:

- кнопка стала красной, а буквы на ней белыми;
- заголовок — крупный и жирный, описание — помельче и посветлее;
- между карточками товаров появились ровные промежутки;
- товары выстроились в три колонки вместо кучи;
- на телефоне те же товары встали в одну колонку;
- при наведении мышкой карточка чуть приподнялась.

Аналогия: одежда, причёска и осанка. Скелет тот же, а впечатление совсем другое.

Ещё точнее — **ремонт в квартире**. Планировка не изменилась: те же комнаты, те же окна. Но после ремонта это другое жильё.

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

Одна строчка — и серая системная кнопка превратилась в красную кнопку с мягкими углами.

► JavaScript — рефлексы браузера

JavaScript отвечает на вопрос «что произойдёт, если человек что-то сделает?» — причём прямо здесь, в браузере, без перезагрузки страницы.

Примеры того, что делает JavaScript:

- нажали «Купить» — счётчик корзины мгновенно стал «3»;
- набрали в поиске «коло» — снизу выпали подсказки: «колодки», «кольцо», «коллектор»;
- кликнули на фото — оно открылось на весь экран;
- на телефоне нажали три полоски — выехало меню;
- начали заполнять форму и забыли телефон — поле подсветилось красным ещё до отправки;
- пролистали страницу вниз — шапка «прилипла» к верху экрана.

Аналогия: рефлекс. Дотронулись до горячего — рука отдёрнулась сама, мозг ещё даже не понял, что случилось.

Ещё точнее — **автоматические двери в магазине**. Никто не звонит директору, чтобы вам открыли. Датчик увидел человека — двери разъехались.

```
button.onclick = () => alert('Товар добавлен в корзину');
```

► PHP — мозг на сервере

А вот это самое важное место, и здесь нужно быть внимательным.

PHP работает **не в браузере**. Он живёт на **сервере** — на чужом компьютере, который стоит где-то в дата-центре и работает круглосуточно. Покупатель до него не дотягивается в принципе.

PHP принимает все серьёзные решения.

Примеры того, что делает PHP:

- проверяет, правильный ли пароль ввели при входе;
- решает, показать человеку кнопку «Админка» или не показывать;
- считает итоговую цену: цена поставщика + наценка + доставка;
- применяет скидку постоянному покупателю — или не применяет;
- записывает заказ и присваивает ему номер;
- отправляет письмо «ваш заказ принят»;
- не даёт продавцу Ахмаду редактировать товары продавца Сафара.

Аналогия: мозг. Или директор магазина — тот, кто решает, а не тот, кто улыбается в зале.

А теперь — правило, которое стоит запомнить прямо сейчас.

JavaScript выполняется **на компьютере покупателя**. Это значит, что покупатель может его открыть, почитать и переписать. Это не сложно — любой браузер позволяет.

PHP выполняется **на вашем сервере**. Покупатель его не видит и изменить не может.

Представьте, что цена товара считается на JavaScript. Тогда покупатель открывает инструменты браузера, меняет 250 на 1 — и оформляет заказ на сомони. Это как если бы в магазине разрешили покупателям самим переклеивать ценники.

Поэтому **цены, скидки, доступы и всё, что связано с деньгами, считает только PHP.**

Никогда — JavaScript. Мы к этому ещё вернёмся не раз, но услышать это лучше в первой главе.

```
if ($user['role'] === 'admin') {  
    echo 'Добро пожаловать в админку';  
}
```

► SQL — разговор с памятью

SQL — это язык, на котором разговаривают с базой данных. Спрашивают и записывают.

Примеры того, что делает SQL:

- найти все тормозные колодки дешевле 300 сомони, которые есть в наличии;
- достать данные одного товара по его номеру;
- добавить новый заказ;
- изменить статус заказа с «новый» на «отправлен»;
- посчитать, на какую сумму продал каждый продавец за месяц;
- найти покупателей, которые не заходили полгода.

Аналогия: разговор с кладовщиком. Вы не идёте на склад сами и не роетесь в коробках. Вы говорите: «дай мне все колодки Bosch, что есть в наличии» — и он приносит список.

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

► Коротко, чтобы уложилось

ИНСТРУМЕНТ	ОТВЕЧАЕТ НА ВОПРОС	ГДЕ РАБОТАЕТ	БЫТОВАЯ АНАЛОГИЯ
HTML	Что здесь есть?	В браузере	Скелет, чертёж дома
CSS	Как это выглядит?	В браузере	Одежда, ремонт
JavaScript	Что будет, если нажать?	В браузере	Рефлексы, автодвери
PHP	Что решаем и что показываем?	На сервере	Мозг, директор
SQL	Где это записано?	В базе данных	Кладовщик, картотека

Если сейчас каша в голове — это абсолютно нормально. Никто не запоминает пять новых понятий с первого раза. Мы к каждому вернёмся отдельной главой, и не по одному разу.

Выше промелькнуло пять кусочков кода, и они наверняка выглядели как шифр. Давайте расшифруем — **каждый** значок.

Запоминать ничего не нужно. Просто прочитайте один раз, чтобы код перестал пугать. Это как впервые посмотреть на ноты: сначала мурашки, потом оказывается, что там всего семь букв.

► HTML: почему всё в «уголках»

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
< и >	«уголки», угловые скобки	Внутри уголков — команда браузеру , а не текст для чтения. На экране их не будет
<h1>	«эйч-один»	Заголовок первого уровня , самый главный на странице. От английского <i>heading</i> — заголовок. Бывают <h1> ... <h6> : чем больше цифра, тем мельче заголовок
</h1>	«закрывающий эйч-один»	Косая черта / означает конец . «Заголовок закончился вот здесь»
<p>	«пи»	Абзац обычного текста. От <i>paragraph</i>
<button>	«баттон»	Кнопка , на которую можно нажать
Текст между тегами		То единственное, что реально увидит человек на экране

Пара «открывающий тег + содержимое + закрывающий тег» называется **элемент**.

Правило простое, как в математике: **что открыли — то надо закрыть**. Забыли закрыть — браузер начнёт фантазировать, и страница поедет.

А что такое **<div>** ?

Это слово вы будете встречать чаще всех остальных, поэтому знакомимся сразу.

`<div>` — это **пустая коробка**. Он ничего не означает и никак не выглядит. На экране его не видно вообще.

Зачем тогда нужен? Чтобы **сложить в него другие элементы** и потом обращаться ко всей группе сразу.

```
<div>
  <h1>Тормозные колодки</h1>
  <p>Цена: 250 сомони</p>
  <button>Купить</button>
</div>
```

Теперь заголовок, цена и кнопка лежат в одной коробке. И можно сказать: «вот эту коробку сделай белой, добавь рамку, скругли углы и отодвинь от соседних». Получится **карточка товара** — та самая, из которых собран любой каталог.

Название `div` — сокращение от английского *division*, «раздел», «отделение». Думайте о нём как о коробке из-под обуви: сама по себе не нужна, но сложить в неё вещи и подписать — очень удобно. Подробно разберём в главе 5.

► CSS: почему фигурные скобки

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>	селектор	Кого красим. Здесь — все кнопки на странице
<code>{ }</code>	фигурные скобки	«Начало и конец списка указаний». Всё внутри относится к кнопке
<code>background</code>	свойство	Что меняем — цвет фона
<code>:</code>	двоеточие	Разделитель: слева что меняем, справа на что
<code>#d92b2b</code>	значение	Цвет в виде кода: <code>#</code> и шесть символов. Здесь — красный
<code>;</code>	точка с запятой	Конец одного указания. Дальше пойдёт следующее
<code>color: white</code>		Цвет текста — белый. Не путайте с <code>background</code> : это фон
<code>border-radius: 8px</code>		Скруглить углы на 8 пикселей

Формула CSS всегда одна и та же:

кого `{` что меняем `:` на что `;` `}`

Выучите её один раз — и любой CSS в мире станет читаемым.

► JavaScript: точки и стрелки

```
button.onclick = () => alert('Товар добавлен в корзину');
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>		Кнопка, которую мы до этого нашли на странице
<code>.</code>	точка	«Возьми у этого вот это». Как «Иван ^{***} . ^{***} телефон» — телефон Ивана
<code>onclick</code>	«он-клик»	Свойство «что делать при нажатии». <i>on click</i> — «по щелчку»
<code>=</code>	равно, присваивание	Положи в левое то, что справа. Это не «равенство» из математики, а команда «запиши»
<code>() =></code>	«стрелочная функция»	«Вот действие , которое надо выполнить». В круглых скобках — место для входных данных, здесь они не нужны
<code>alert(...)</code>	«алерт»	Готовая команда браузера: показать всплывающее окошко
<code>'...'</code>	кавычки	Внутри кавычек — текст , а не команда. Компьютер не пытается его выполнить
<code>;</code>	точка с запятой	Конец строки-команды

► PHP: доллары и условия

```
if ($user['role'] === 'admin') {
    echo 'Добро пожаловать в админку';
}
```

Вот здесь появилось `if` — пожалуй, самое важное слово во всём программировании. Оно есть в каждом языке, и работает везде одинаково.

ЧТО ВИДИМ КАК ЧИТАЕТСЯ ЧТО ЭТО ЗНАЧИТ

<code>if</code>	«иф», по-английски «если»	Проверка. «Если условие верно — сделай то, что в фигурных скобках. Если нет — просто пропусти»
<code>()</code>	круглые скобки после <code>if</code>	Внутри — условие , которое проверяем
<code>\$</code>	доллар	В PHP так помечают переменную — коробочку с данными. <code>\$user</code> — «коробочка с пользователем»
<code>\$user['role']</code>		Достать из коробочки <code>\$user</code> поле <code>role</code> — роль. Квадратные скобки: «возьми вот эту часть»
<code>===</code>	«строго равно»	Сравнение: «а точно ли слева то же самое, что справа?» Три знака, не один
<code>'admin'</code>		Текст, с которым сравниваем
<code>{ }</code>	фигурные скобки	Тело условия — что делать, если проверка прошла
<code>echo</code>	«эхо»	Команда PHP: выведи это на страницу
<code>;</code>		Конец команды

Один `=` или три `===` ? Самая частая ошибка новичка.

- `=` — это **записать**. `$x = 5` значит «положи пятёрку в коробочку x».
- `===` — это **спросить**. `$x === 5` значит «а в коробочке правда пятёрка?»

Первое — действие, второе — вопрос. Перепутаете — программа вместо проверки молча запишет значение, и ошибку вы будете искать долго.

А где же `else` ?

`else` — «иначе» — это вторая половина `if`. Читается ровно как в жизни:

```
if ($user['role'] === 'admin') {
    echo 'Добро пожаловать в админку';
} else {
    echo 'Вам сюда нельзя';
}
```

Дословно: **если** роль администратор — покажи приветствие, **иначе** — откажи. Одно из двух выполнится обязательно, третьего не дано.

Вы пользуетесь этой конструкцией каждый день, просто не называете её так:

- если дождь — беру зонт, **иначе** — не беру;
- если денег хватает — покупаю, **иначе** — откладываю;
- если на светофоре зелёный — иду, **иначе** — стою.

Вот и всё программирование. Подробно — в главе 21.

► SQL: запрос как обычная фраза

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

SQL специально сделали похожим на английское предложение. Читайте по кусочкам:

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
SELECT	«селект» — выбрать	Какие колонки нам нужны
name, price		Название и цена. Запятая — «и ещё»
FROM	«фром» — из	Из какой таблицы берём
products		Таблица товаров
WHERE	«вэа» — где, при условии	Какие строки берём, а какие пропускаем
price < 300		Цена меньше 300
AND	«энд» — и	Оба условия должны выполняться одновременно
in_stock = 1		Есть в наличии (1 — да, 0 — нет)
;		Конец запроса

Целиком получается: «Выбери название и цену из таблицы товаров, где цена меньше 300 и товар в наличии». Практически человеческая фраза.

► Что общего у всех пяти языков

А теперь приятное. Присмотритесь к таблицам выше — значки-то повторяются:

ЗНАЧОК

ПОЧТИ В ЛЮБОМ ЯЗЫКЕ ОЗНАЧАЕТ

`{ }`

«блок»: вот начало группы, вот конец

`( )`

«сюда кладут данные»: условие или входные значения

`;`

«команда закончилась»

`' '` или `" "`

«внутри текст, выполнять его не надо»

`//` или `#`

комментарий: заметка для человека, компьютер её не читает

Разобравшись один раз, вы будете узнавать их везде — хоть в Python, хоть в Java, хоть в языке, который придумают через десять лет. Языков много, а привычки у них общие.

РАЗБИРАЕМСЯ

А теперь всё вместе

Давайте проследим, что происходит, когда покупатель открывает страницу товара.

Прочитайте медленно — это самая важная схема в книге, всё остальное будет крутиться вокруг неё.

1. Покупатель нажимает ссылку «Тормозные колодки».
2. Браузер отправляет на сервер запрос: «дайте мне страницу товара номер 42».
3. На сервере просыпается PHP и думает: «сорок второй, значит. Надо узнать подробности».
4. PHP составляет запрос на SQL и отправляет его в базу данных: `SELECT * FROM products WHERE id = 42`.
5. База отвечает: «Колодки тормозные, Bosch, 250 сомони, на складе 7 штук».
6. PHP берёт эти данные и строит из них HTML: название вставляет в заголовок, цену — в ценник, картинку — в блок картинки.
7. Готовый HTML улетает обратно в браузер.
8. Браузер читает HTML, применяет CSS — и страница из чёрно-белого текста превращается в нормальный магазин.
9. Подключается JavaScript и начинает следить за кнопкой «Купить».

Что происходит, когда покупатель открывает страницу товара

Три зоны, пять технологий — и меньше секунды на всё



Всё это заняло меньше секунды. Пять технологий, и каждая сделала ровно свою часть работы.

Разделение обязанностей — вот главная идея всей веб-разработки. HTML не красит. CSS не считает. JavaScript не лезет в базу. PHP не рисует. Каждый занят своим делом и не мешает соседям.

Звучит как формальность, но именно поэтому сайты вообще можно чинить. Сломалась внешность — идёте в CSS и точно знаете, что база данных тут ни при чём.

РАЗБИРАЕМСЯ

Что именно мы построим

Не «примерчик на три кнопки». Настоящий интернет-магазин — такой, каким пользуются живые люди и в котором ходят реальные деньги.

Для покупателя:

- каталог запчастей с поиском по названию и по артикулу;
- фильтры: бренд, цена, наличие;
- карточка товара с фото, описанием и характеристиками;
- корзина: добавить, изменить количество, удалить;
- оформление заказа с адресом и телефоном;
- личный кабинет с историей заказов и их статусами.

Для продавца — потому что магазин будет не только ваш, а маркетплейс:

- регистрация продавца и его страница;
- выкладка своих товаров;
- свои заказы и работа с ними;
- баланс: сколько заработано и сколько уже выплачено.

Для администратора:

- модерация: проверять товары продавцов до публикации;
- управление всеми заказами и статусами;
- расчёт комиссии магазина;
- реестр выплат продавцам.

И взрослые вещи, которых обычно нет в учебниках:

- подключение внешнего API поставщика запчастей;
- пересчёт цены из рублей в сомони по живому курсу, плюс наценка и доставка;
- защита от взлома: SQL-инъекции, XSS, кража сессии;
- выкладка сайта на настоящий хостинг, под настоящим доменом, с замочком HTTPS.

Дойти до конца — это работа на несколько месяцев. Но каждая глава заканчивается чем-то, что можно открыть в браузере и показать. Ощущение «у меня получилось» будет приходить регулярно, а не один раз в самом конце.

РАЗБИРАЕМСЯ

Почему PHP, а не что-то модное

Отвечу честно, потому что вопрос вы всё равно услышите. В интернете любят повторять, что «PHP устарел».

PHP выбран не потому, что он самый модный. А потому, что для **первого** сайта он самый удобный:

- **Результат виден сразу.** Создали файл, открыли в браузере — работает. Не нужно ставить пятьдесят пакетов и настраивать сборщик, чтобы вывести на экран слово «привет».
- **Дешёвый хостинг.** PHP работает на хостинге за 3–5 долларов в месяц. Для первого настоящего проекта это важно: сайт можно показать людям, не разорившись.
- **Огромная база готовых ответов.** Какую бы проблему вы ни встретили, её уже кто-то решил и описал.
- **На нём работает больше половины сайтов в мире.** Один только WordPress — это примерно четверть всего интернета.

Но главное вот что. **Навык — не в языке.**

Когда вы научитесь думать «пришёл запрос → проверили права → сходили в базу → собрали ответ», вы пересядете на Python или Node.js за пару недель. Синтаксис разный, мышление одно и то же.

Языки приходят и уходят. Умение раскладывать задачу на части остаётся.

ГРАБЛИ

Грабли

Четыре мысли, которые чаще всего мешают новичкам. Если поймаете себя на любой из них — вспомните эту страницу.

«Сначала выучу всё, потом начну делать». Не сработает, проверено многими. Знания без практики выветриваются за неделю. Правильный порядок обратный: делаем маленькую вещь → упираемся в незнание → узнаём ровно столько, сколько нужно прямо сейчас → делаем дальше. Скучно, зато держится.

«Надо сразу выбрать самый лучший язык». Лучшего языка не существует, есть подходящий для задачи. Споры «PHP против Python» для новичка — это спор о том, каким молотком забивать первый в жизни гвоздь. Забейте хоть каким, потом разберётесь.

«Настоящие программисты знают всё наизусть». Нет. Разработчик с пятнадцатью годами опыта гуглит синтаксис по десять раз в день, и это нормально. Ценность не в памяти, а в умении разбить задачу на части и понять, что именно искать.

«Я не математик, значит, не потяну». Для веб-разработки нужна арифметика уровня седьмого класса: сложить цены, посчитать проценты, поделить на количество. Всё. Высшая математика понадобится, если вы пойдёте в машинное обучение — но это совсем другая история.

Пять упражнений. Ни в одном не нужен компьютер с редактором — только голова и браузер. Не пропускайте: именно на задачах прочитанное превращается в понятное.

Задача 1.1. Откройте любой знакомый сайт — маркетплейс, новости, банк. Выпишите пять вещей, которые вы **видите** (это работа фронтенда). Потом пять вещей, которые происходят **невидимо**: проверка пароля, расчёт доставки, сохранение заказа (это бэкенд).

Задача 1.2. Для каждой строки определите, кто отвечает — HTML, CSS, JavaScript, PHP или SQL:

1. Кнопка стала синей вместо серой.
2. На странице появился второй заголовок.
3. При наборе букв в поиске выпадают подсказки.
4. Проверка, что пароль при входе правильный.
5. Достать из базы 20 самых дешёвых товаров.
6. Меню на телефоне схлопнулось в три полоски.
7. Заказ записался, и ему присвоился номер.
8. Между карточками товаров появились отступы.

Задача 1.3. Объясните вслух — правда вслух, это работает — своими словами разницу между фронтендом и бэкендом. Уложитесь в два предложения и не используйте сами слова «фронтенд» и «бэкенд».

Задача 1.4. Почему скидку постоянного покупателя нельзя считать на JavaScript? Ответ — одна строчка.

Задача 1.5. Нажмите на любом сайте правую кнопку мыши и выберите «Посмотреть код страницы». Откроется HTML, который прислал сервер. Найдите в нём заголовок страницы (ищите `<h1>` или `<title>`). Если сейчас это выглядит как сплошная каша — так и должно быть. К главе 8 вы будете читать её свободно.

Задача 1.6. Придумайте три своих аналогии для тройки «витрина — подсобка — журнал». Не из книги, а свои: школа, аэропорт, парикмахерская — что угодно. Чем больше своих примеров, тем прочнее держится в голове.

Ответы — в Приложении Г.

В БОЮ В бою

А теперь заглянем в настоящий проект. В корне этого репозитория лежит код работающего магазина `autodoc.tj`. Открывать файлы пока не нужно — просто посмотрите на названия папок. Вы уже можете угадать, что где:

ПАПКА	ЧТО ВНУТРИ	КАКАЯ ЭТО ЗОНА
<code>assets/css/</code>	Файлы оформления	Фронтенд (CSS)
<code>pages/</code>	Страницы сайта	Фронтенд + бэкенд
<code>includes/</code>	Общая логика, которую используют все страницы	Бэкенд (PHP)
<code>sql/</code>	Структура базы данных	База данных
<code>admin/</code>	Админка	Бэкенд, «подсобка»
<code>seller/</code>	Кабинет продавца	Бэкенд, «подсобка»
<code>api/</code>	Точки, куда обращается JavaScript	Стык фронта и бэка

Узнаёте? Это тот самый магазин из начала главы. Торговый зал, подсобка и складской журнал — только разложенные по папкам.

Через шестьдесят глав вы откроете любой из этих файлов и будете понимать, что там написано, строчка за строчкой.

- Сайт — это **торговый зал** (фронтенд), **подсобка** (бэкенд) и **складской журнал** (база данных). Всегда, у любого сайта.
- **HTML** — скелет, **CSS** — внешность, **JavaScript** — рефлекс в браузере, **PHP** — мозг на сервере, **SQL** — разговор с памятью.
- JavaScript выполняется у покупателя и переписывается им за минуту. PHP выполняется у вас. Поэтому деньги и доступы считает **только PHP**.
- `if` — это «если», `else` — «иначе». `=` записывает, `===` спрашивает. `<div>` — пустая коробка для группировки.
- **Fullstack** — тот, кто работает во всех трёх зонах. Это вы через шестьдесят глав.
- Учиться нужно от задачи к теории, а не наоборот.

В следующей главе разберёмся, что вообще происходит между моментом, когда вы вводите адрес сайта, и моментом, когда на экране появляется картинка. Спойлер: там интересно.

Как работает интернет

ЗАЧЕМ ЭТО**Зачем эта глава**

Вы вводите в браузере адрес, жмёте Enter — и через полсекунды видите страницу. Кажется, что там нечего объяснять: нажал и появилось.

На самом деле за эти полсекунды происходит примерно десять отдельных событий, и каждое из них однажды у вас сломается. Сайт не открывается, картинки не грузятся, форма отпавилась «в нукуда», на телефоне работает, а на компьютере нет.

Человек, который не понимает эту цепочку, в такие моменты просто разводит руками.

Человек, который понимает, спокойно спрашивает себя: «так, а на каком шаге сломалось?» — и находит причину за пять минут.

Поэтому потратим главу на то, что происходит между вашим Enter и картинкой на экране. Кода снова не будет. Будет понимание, без которого дальше всё превратится в магию.

РАЗБИРАЕМСЯ**Интернет — это почта**

Самое вредное представление об интернете — «где-то там облако, и в нём всё лежит». Давайте заменим его на правильное.

Интернет — это почтовая служба. Огромная, быстрая, но обычная почта.

Есть отправитель. Есть получатель с адресом. Есть письмо с вопросом и письмо с ответом. Есть курьеры, которые везут письма по маршруту. Всё, никакой магии.

Когда вы открываете сайт, происходит ровно это:

1. Вы (браузер) пишете письмо: «здравствуйте, пришлите мне, пожалуйста, главную страницу».
2. Письмо едет по адресу.
3. Там его читает сервер и готовит ответ.
4. Ответ приезжает обратно.
5. Браузер разворачивает конверт и показывает содержимое.

Одно письмо туда, одно обратно. Это называется **запрос и ответ** (по-английски *request* и *response*). Запомните эту пару — она будет встречаться в книге сотни раз.

Что происходит между вашим Enter и картинкой на экране

Одно письмо туда, одно обратно — и десяток шагов между ними

- 1 Разбирает адрес** **БРАУЗЕР**
Делит `https://autodoc.tj/catalog` на протокол, домен и путь
- 2 Спрашивает номер дома** **DNS**
«Какой IP у autodoc.tj?» — «185.12.94.7». Справочник интернета
- 3 Стучится и договаривается о шифровании** **БРАУЗЕР**
Тот самый замочек в адресной строке появляется здесь
- 4 Отправляет запрос** **БРАУЗЕР**
`GET /catalog HTTP/1.1` — «покажи, пожалуйста, каталог»
- 5 Думает** **СЕРВЕР**
PHP идёт в базу через SQL, забирает товары и собирает HTML. 20–200 мс
- 6 Отвечает** **СЕРВЕР**
`200 OK` и готовый HTML. Или `404`, если такого адреса нет
- 7 Рисует страницу и просит остальное** **БРАУЗЕР**
За каждым стилем, скриптом и картинкой уходит отдельный запрос. Их бывает 30–80

Где обычно ломается: шаг 2 — домен куплен, но не настроен, сайт «не открывается». Шаг 5 — ошибка в вашем коде, приходит **500**. Шаг 6 — опечатка в адресе, приходит **404**.

РАЗБИРАЕМСЯ

Клиент и сервер

Два слова, которые нужно развести раз и навсегда.

Клиент — тот, кто просит. В нашем случае это **браузер**: Chrome, Safari, Firefox, Яндекс.Браузер — любой. Он живёт на вашем телефоне или компьютере.

Сервер — тот, у кого просят. Это компьютер, который включён круглосуточно и умеет отвечать на письма.

И вот тут первое открытие для многих: **сервер** — это обычный компьютер. Не волшебный ящик. У него есть процессор, память, диск и операционная система. Отличий всего три:

- он не выключается;
- у него нет монитора и клавиатуры — им управляют по сети;
- он стоит не дома, а в дата-центре, где хороший интернет и не пропадает электричество.

Ваш ноутбук тоже может стать сервером. Мы это сделаем уже в главе 4: запустим на нём маленький сервер и откроем собственный сайт. Никакой мистики.

Кстати, «клиент» и «сервер» — это роли, а не предметы. Один и тот же компьютер может быть сервером для одних и клиентом для других. Наш магазин будет сервером для покупателей и клиентом, когда пойдёт спрашивать цены у поставщика.

РАЗБИРАЕМСЯ

Что происходит, когда вы жмёте Enter

Теперь подробно. Вы набрали `autodoc.tj` и нажали Enter. Поехали.

► Шаг 1. Браузер разбирает адрес

Адрес сайта — это не одно слово, а несколько частей. Возьмём полный вид:

```
https://autodoc.tj/catalog/brakes?page=2
```

КУСОК	КАК НАЗЫВАЕТСЯ	ЧТО ЗНАЧИТ
<code>https</code>	протокол	На каком языке разговаривать. <code>https</code> — «по-человечески, но зашифрованно»
<code>://</code>	разделитель	Просто знак, что дальше идёт адрес
<code>autodoc.tj</code>	домен	Имя дома, куда едем
<code>/catalog/brakes</code>	путь	Какая именно комната в доме нужна
<code>?page=2</code>	параметры	Уточнение: «вторая страница»

Браузер это всё раскладывает и понимает, куда собирается.

► Шаг 2. Браузер узнаёт настоящий адрес — DNS

Вот тут интересное. Компьютеры не умеют находить друг друга по именам вроде `autodoc.tj`. Они работают с числами — IP-адресами, например `185.12.94.7`.

Имя нужно людям. Число нужно машинам.

Поэтому существует DNS — телефонный справочник интернета. Браузер спрашивает: «какой номер у `autodoc.tj`?» — и получает: «185.12.94.7».

Аналогия проще некуда. Вы же не помните номер телефона мамы — вы открываете контакты и жмёте «Мама». Контакты сами подставляют цифры. DNS — это контакты интернета.

⚠ Почему это важно знать: если сайт «не открывается», очень часто виноват именно DNS. Домен купили, но не настроили — и справочник просто не знает, куда вас отправить. В главе 58 мы это настроим руками.

► Шаг 3. Браузер стучится в дверь

Зная IP-адрес, браузер устанавливает **соединение** с сервером. Как позвонить и дождаться, пока снимут трубку.

Если адрес начинался с `https`, то в этот момент собеседники ещё и договариваются о шифровании: «давай дальше разговаривать так, чтобы посторонние не поняли». Именно поэтому в браузере появляется замочек.

Чем `https` отличается от `http` на бытовом уровне:

- `http` — открытка. Её может прочитать любой почтальон по дороге.
- `https` — запечатанный конверт. Видно, что письмо едет, но не видно, что внутри.

Пароли и номера карт по открыткам не отправляют. Поэтому сегодня `https` обязателен.

► Шаг 4. Браузер отправляет запрос

Наконец само письмо. Выглядит оно примерно так:

```
GET /catalog/brakes HTTP/1.1
Host: autodoc.tj
User-Agent: Chrome
Accept-Language: ru
```

Не пугайтесь, читается легко:

СТРОКА	ЧТО ГОВОРИТ
GET	«Дай посмотреть». Это метод — тип просьбы
/catalog/brakes	Что именно дать
HTTP/1.1	На какой версии языка говорим
Host:	В какой дом стучимся
User-Agent:	Кто пришёл: браузер представляется
Accept-Language:	«Я предпочитаю русский»

Строки после первой называются **заголовками** — это уточнения к просьбе.

Методов запроса всего **несколько**, и два из них вам понадобятся постоянно:

МЕТОД	СМЫСЛ	ПРИМЕР ИЗ ЖИЗНИ МАГАЗИНА
GET	«Покажи»	Открыть каталог, посмотреть товар
POST	«Прими и запиши»	Оформить заказ, отправить форму
PUT	«Замени целиком»	Обновить карточку товара
DELETE	«Удали»	Удалить товар из корзины

Разницу GET и POST разберём подробно в главе 24, когда будем делать формы. Пока запомните так: **GET смотрит, POST меняет**.

► Шаг 5. Сервер думает

Письмо дошло. Сервер его прочитал и понял: просят страницу каталога тормозных колодок.

И вот здесь просыпается всё то, о чём мы говорили в первой главе. Запускается **PHP**. Он идёт в **базу данных** с запросом на **SQL**, получает список товаров, подставляет их в шаблон и собирает **HTML**.

Занимает это обычно от 20 до 200 миллисекунд. Если дольше — пользователи начинают уходить, и в главе 30 мы будем разбираться, почему тормозит.

► Шаг 6. Сервер отвечает

Ответ выглядит так:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html>
  <head><title>Тормозные колодки</title></head>
  <body>... </body>
</html>
```

Сверху — служебная часть, снизу, после пустой строки, — сам HTML.

200 OK — это **код ответа**. Сервер всегда сообщает, чем закончилось дело. Вот те коды, которые вы будете видеть каждый день:

КОД	ЧТО ОЗНАЧАЕТ	КОГДА ВЫ ЕГО ВСТРЕТИТЕ
200	Всё хорошо, вот страница	Обычная работа
301 / 302	Переехали, идите по другому адресу	Перенаправление, например с <code>http</code> на <code>https</code>
403	Вижу вас, но не пущу	Полез в админку без прав
404	Такой страницы нет	Опечатка в адресе, удалённый товар
500	Сервер сам сломался	Ошибка в вашем коде. Самый частый гость новичка
503	Сервер перегружен	Слишком много посетителей разом

Запомните различие, оно экономит часы: **404** — **вы ошиблись адресом**. **500** — **ошиблись вы, но в коде**. Первое лечится ссылкой, второе — чтением логов (глава 56).

► Шаг 7. Браузер рисует страницу

HTML приехал. Браузер читает его сверху вниз и строит из него страницу.

Но HTML — это только текст. По дороге ему встречаются ссылки на другие файлы: стили, картинки, скрипты. И за каждым файлом браузер **отправляет отдельный запрос**.

Вот это открытие обычно удивляет: одна страница — это не одно письмо, а **десятки**.

Запрос 1 → сама страница (HTML)
Запрос 2 → файл стилей (CSS)
Запрос 3 → скрипт (JavaScript)
Запрос 4 → логотип
Запрос 5 → фото товара №1
Запрос 6 → фото товара №2
... и так далее

Обычная страница магазина — это 30-80 запросов. Поэтому сайты и «весят»: не текст тяжёлый, а картинки, которых много.

► Шаг 8. Оживает JavaScript

Последним включается JavaScript и начинает следить за действиями: клики, ввод, прокрутка. Страница из картинки превращается в живую.

Собрали много новых слов. Разложим по полочкам — это ваш словарь на все оставшиеся главы.

СЛОВО	КАК ЧИТАЕТСЯ	ЧТО ЭТО ПРОСТЫМИ СЛОВАМИ
HTTP	«эйч-ти-ти-пи»	Язык, на котором браузер разговаривает с сервером. Правила переписки
HTTPS	«эйч-ти-ти-пи-эс»	То же самое, но зашифрованное. S — от <i>secure</i> , «безопасный»
Запрос (request)		Письмо от браузера к серверу: «дай мне вот это»
Ответ (response)		Письмо обратно: «вот, держи» или «не могу, вот почему»
Клиент		Тот, кто просит. Обычно браузер
Сервер		Тот, у кого просят. Компьютер, который всегда включён
IP-адрес	«ай-пи»	Номер компьютера в сети: 185.12.94.7
Домен		Человеческое имя сайта: autodoc.tj
DNS	«ди-эн-эс»	Справочник, который переводит имя в номер
Хостинг		Услуга: вам сдают в аренду место на сервере
Заголовки (headers)		Уточнения к письму: кто я, что понимаю, на каком языке
GET	«гет»	Метод «покажи»
POST	«пост»	Метод «прими и запиши»
Код ответа		Число-итог: 200 — хорошо, 404 — нет такого, 500 — сломалось
Порт		Номер двери в доме. У http дверь 80, у https — 443

Про порт чуть подробнее, потому что он вам скоро встретится. Если IP-адрес — это дом, то порт — квартира в этом доме. На одном компьютере может работать несколько программ, и каждая слушает свою дверь. Когда в главе 4 мы запустим сервер на своём компьютере, адрес будет выглядеть как `localhost:8000` — вот `8000` и есть номер двери.

А `localhost` — это «мой собственный компьютер». Всегда. На любой машине. Его IP — `127.0.0.1`, и он тоже всегда один и тот же.

НА ЭКРАНЕ

Посмотрите на это своими глазами

Всё описанное можно увидеть прямо сейчас, без единой строчки кода.

1. Откройте любой сайт в Chrome.
2. Нажмите F12 (на Mac — `Cmd+Option+I`). Откроется панель разработчика.
3. Перейдите на вкладку **Network** («Сеть»).
4. Обновите страницу клавишей F5.

Вы увидите список — это все те самые запросы. Каждая строка: имя файла, код ответа, размер, сколько миллисекунд занял.

Пощёлкайте по строкам. Справа откроются заголовки запроса и ответа — те же самые, что мы разбирали выше, только настоящие.

Это не учебная симуляция. Это реальная переписка вашего браузера с сервером. Профессиональные разработчики смотрят в эту вкладку каждый день.

ГРАБЛИ

Грабли

«Сайт лежит» — а на самом деле нет. Прежде чем паниковать, откройте его в другом браузере, с телефона по мобильному интернету и в режиме инкогнито. Очень часто «лежит» только у вас: кэш, DNS, провайдер, старая версия страницы.

Кэш — ваш главный обманщик. Браузер запоминает файлы, чтобы не качать их дважды. Полезно для скорости, вредно при разработке: вы поправили стили, а браузер показывает старые. Лечится жёстким обновлением: `Ctrl+F5` (на Mac `Cmd+Shift+R`). Запомните эту комбинацию — она сэкономит вам десятки часов недоумения.

Путать 404 и 500. 404 — «я не нашёл такой адрес». 500 — «я нашёл, но при выполнении всё упало». Это разные проблемы и разные места поиска.

Думать, что **https** защищает сайт от взлома. Не защищает. **https** шифрует только дорогу между браузером и сервером. Если в вашем коде дыра, шифрование канала не поможет никак. Безопасностью займёмся в главе 36.

Считать, что домен и хостинг — одно и то же. Домен — это имя. Хостинг — это место, где живёт сайт. Покупаются отдельно, часто у разных компаний. Как отдельно взять номер телефона и отдельно снять квартиру.

ЗАДАЧИ

Задачи

Задача 2.1. Разберите адрес на части — назовите протокол, домен, путь и параметры:

`https://autodoc.tj/search?q=kolodki&brand=bosch`

Задача 2.2. Какой код ответа вы получите в каждом случае?

1. Открыли главную страницу работающего сайта.
2. Ошиблись в адресе на одну букву.
3. Программист забыл точку с запятой, и PHP упал.
4. Зашли в админку, не будучи администратором.

Задача 2.3. Откройте панель разработчика (F12 → Network) на любимом сайте и обновите страницу. Сколько всего запросов ушло? Какой файл оказался самым тяжёлым? Запишите числа — в главе 57 мы вернёмся к ним, когда будем ускорять свой сайт.

Задача 2.4. Объясните своими словами, что такое DNS. Одно предложение, без слова «DNS».

Задача 2.5. Что произойдёт, если DNS сломается, а сайт при этом работает? Сможете ли вы попасть на сайт вообще?

Задача 2.6. Почему форму оформления заказа отправляют методом POST, а не GET? Подсказка: подумайте, что окажется в адресной строке браузера.

Ответы — в Приложении Г.

В коде `autodoc.tj` эта глава встречается буквально везде.

Загляните в любой файл в папке `api/` — например `api/vin_order_request.php`. Первые строки почти каждого такого файла проверяют метод запроса:

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {  
    http_response_code(405);  
    exit;  
}
```

Дословно: «если пришли не методом POST — верни код 405 („метод не разрешён“) и заканчивай». Это защита: страницу, которая создаёт заказ, нельзя открывать простой ссылкой.

Через двадцать глав вы будете писать такие проверки не задумываясь.

- Интернет — это почта. **Запрос** туда, **ответ** обратно.
- **Клиент** просит (браузер), **сервер** отвечает. Сервер — обычный компьютер, который не выключают.
- DNS переводит имя `autodoc.tj` в номер `185.12.94.7`. Как контакты в телефоне.
- HTTP — язык переписки. **GET** смотрит, **POST** записывает.
- **Коды ответа**: 200 — хорошо, 404 — нет адреса, 500 — сломался код, 403 — не пушу.
- Одна страница — это десятки запросов: HTML, стили, скрипты, каждая картинка.
- **F12** → **Network** показывает всю эту переписку вживую. Пользуйтесь.
- **Ctrl+F5** обновляет страницу мимо кэша. Запомните прямо сейчас.

В следующей главе перестанем читать и наконец возьмём инструменты в руки.

Инструменты: редактор, терминал, браузер

ЗАЧЕМ ЭТО

Зачем эта глава

Хорошая новость: чтобы делать сайты, не нужен мощный компьютер и не нужны платные программы. Нужны три вещи, и все три бесплатные.

Плохая новость: одна из них — **терминал**, то самое чёрное окно с буквами. Именно на нём спотыкается больше всего новичков. Не потому, что он сложный, а потому, что выглядит недружелюбно.

В этой главе мы поставим всё необходимое и — главное — подружимся с терминалом. К концу главы вы будете уверенно перемещаться по папкам и запускать программы, не трогая мышку.

РАЗБИРАЕМСЯ

Три инструмента, и всё

ИНСТРУМЕНТ	ЧТО ДЕЛАЕТ	АНАЛОГИЯ ИЗ МАСТЕРСКОЙ
Редактор кода	В нём вы пишете код	Верстак, за которым работаете
Терминал	В нём вы запускаете программы	Пульт управления станками
Браузер	В нём вы смотрите результат	Зеркало: сразу видно, что получилось

Всё. Ни «мощного железа», ни платных подписок. Подойдёт любой компьютер, купленный за последние десять лет.

► Почему не Word и не «Блокнот»

Первый вопрос новичка: а нельзя писать код в Word? Нельзя, и вот почему.

Word — это **текстовый процессор**. Он не хранит просто буквы, он хранит оформление: шрифты, размеры, отступы. И когда вы сохраните файл, туда попадёт куча невидимого мусора. Браузер такое не поймёт.

Код — это **чистый текст**. Только символы, ничего больше.

«Блокнот» из Windows чистый текст умеет, но он совершенно ничем не помогает. Это как строить дом одним молотком: теоретически можно, практически — мучение.

► Что ставить: Visual Studio Code

Берите VS Code — бесплатный редактор от Microsoft. Он стал стандартом: на нём работают миллионы разработчиков, под него есть всё на свете.

Скачивается с code.visualstudio.com, ставится как обычная программа, работает на Windows, macOS и Linux.

► Что редактор делает за вас

Вот почему в нём в разы приятнее, чем в блокноте:

- **Подсвечивает синтаксис.** Теги одним цветом, текст другим, ошибки третьим. Глаз сам находит нужное место.
- **Закрывает скобки.** Написали `<div>` — редактор сам поставит `</div>`. Помните правило «что открыли, то закрой»? Он будет следить за этим вместо вас.
- **Подсказывает.** Начали писать `back` — предложит `background`, `background-color` и остальные варианты.
- **Показывает ошибки до запуска.** Забыли кавычку — подчеркнёт красным сразу.
- **Ищет по всему проекту.** Где у меня в тридцати файлах написано `price`? Одна команда.

► Три настройки, которые стоит сделать сразу

Откройте настройки (Ctrl+, или Cmd+,) и поищите по названию:

1. **Word Wrap** → включить (**on**). Длинные строки будут переноситься, а не уезжать за край экрана.
2. **Font Size** → 15 или 16. Глаза скажут спасибо, а вы просидите за кодом много часов.
3. **Auto Save** → **afterDelay**. Файл сохраняется сам. Половина «загадочных ошибок» у новичков — это несохранённый файл.

► Два расширения, которые пригодятся

В VS Code слева есть иконка с кубиками — это магазин расширений. Поставьте:

- **PHP Intelephense** — умные подсказки по PHP.
- **Russian Language Pack** — если хочется интерфейс на русском.

Остальное — потом, по мере надобности. Не увлекайтесь: новички часто ставят тридцать расширений вместо того, чтобы писать код.

РАЗБИРАЕМСЯ

Инструмент второй: терминал

Вот к нему и подошли. Давайте сразу снимем страх.

► Что это вообще такое

Терминал — это способ управлять компьютером словами, а не мышкой. Всё.

Вы уже умеете управлять мышкой: открыть папку, скопировать файл, запустить программу. В терминале вы делаете то же самое, но пишете команду.

Зачем, если мышкой привычнее? Четыре причины:

1. Многие программы просто не имеют окон. Тот же PHP-сервер запускается только командой.
2. Быстрее. Одна строка вместо десяти кликов.
3. Повторяемо. Команду можно записать, отдать коллеге, вставить в инструкцию.
4. На сервере другого способа нет. У сервера нет монитора — только терминал. Когда в главе 57 будем выкладывать сайт, работать придётся именно так.

► Где его найти

СИСТЕМА	КАК ОТКРЫТЬ
Windows	Пуск → напечатать PowerShell → Enter
macOS	Cmd+Пробел → напечатать Терминал → Enter
Linux	Ctrl+Alt+T
Прямо в VS Code	Меню Терминал → Новый терминал, или Ctrl+`

Последний способ — самый удобный. Терминал открывается прямо под кодом, и не нужно прыгать между окнами.

► Как читать эту непонятную строчку

Открыли терминал и увидели что-то вроде:

```
firdavs@MacBook ~/projects %
```

Расшифровываем:

ЧАСТЬ	ЧТО ЗНАЧИТ
firdavs	ваше имя пользователя
MacBook	имя компьютера
~/projects	где вы сейчас находитесь — самое важное
% или \$ или >	приглашение: «пиши команду»

Ключевая мысль: терминал всегда находится в какой-то папке. Как вы, открывая проводник, всегда стоите в какой-то папке. Команды выполняются относительно неё.

Значок ~ означает вашу домашнюю папку. На Windows это C:\Users\Firdavs, на Mac — /Users/firdavs.

► Семь команд, которых хватит надолго

Правда семь. Не сто, не тридцать — семь.

КОМАНДА	РАСШИФРОВКА	ЧТО ДЕЛАЕТ
<code>pwd</code>	<i>print working directory</i>	Где я сейчас? Показывает полный путь
<code>ls</code>	<i>list</i>	Что здесь лежит? Список файлов и папок
<code>cd имя</code>	<i>change directory</i>	Зайти в папку
<code>cd ..</code>		Подняться на уровень выше
<code>mkdir имя</code>	<i>make directory</i>	Создать папку
<code>touch имя</code>		Создать пустой файл
<code>clear</code>		Очистить экран, когда намусорили

На Windows в PowerShell работают все эти команды, кроме `touch`. Вместо него: `New-Item имя.txt`.

► Три приёма, которые экономят время

1. Клавиша Tab дописывает за вас. Начните печатать `cd proj` и нажмите Tab — терминал сам допишет `projects`. Это не только быстрее, но и защищает от опечаток.
2. Стрелка вверх достаёт прошлые команды. Нажмите ↑ — появится предыдущая команда. Ещё раз — позапрошлая.
3. Ctrl+C прерывает. Запустили что-то и хотите остановить — Ctrl+C. Работает почти везде. Понадобится, когда будем гасить наш сервер.

► Потренируйтесь прямо сейчас

Не читайте дальше — откройте терминал и наберите по очереди:

```
pwd
ls
mkdir moi-magazin
cd moi-magazin
pwd
ls
cd ..
```

Что произошло: посмотрели, где мы → посмотрели содержимое → создали папку → зашли в неё → убедились, что вошли → увидели, что она пустая → вышли обратно.

Всё. Вы только что поработали в терминале. Ничего страшного не случилось.

РАЗБИРАЕМСЯ

Инструмент третий: браузер

Браузер у вас уже есть. Но нужно узнать про него кое-что новое.

Возьмите **Chrome** или **Firefox** для разработки. Не потому, что они лучше как браузеры, а потому, что у них лучшие инструменты разработчика и вся документация написана под них.

► Панель разработчика — ваш второй дом

Нажмите F12 (на Mac **Cmd+Option+I**). Панель, которую вы уже открывали в прошлой главе. Вот вкладки, которые понадобятся:

ВКЛАДКА ДЛЯ ЧЕГО

Elements	Смотреть HTML страницы и менять его на лету . Можно поменять цвет кнопки и сразу увидеть результат
Console	Ошибки JavaScript. Первое место, куда смотреть, когда «ничего не работает»
Network	Все запросы: что загрузилось, что нет, сколько заняло
Sources	Исходные файлы страницы

Совет, который экономит нервы: приучитесь держать консоль открытой, когда пишете JavaScript. Половина проблем в ней прямым текстом объяснена — просто туда никто не смотрит.

Пока ставить не нужно, но чтобы вы знали, что впереди:

ЧТО	ЗАЧЕМ	В КАКОЙ ГЛАВЕ
PHP	Чтобы работал серверный код	Глава 19
MySQL	База данных	Глава 26
Git	Хранить историю изменений и не терять работу	Глава 53

Всё это ставится одним пакетом — **XAMPP** (Windows/Mac) или **OpenServer** (Windows). Один установщик кладёт сразу PHP, MySQL и веб-сервер. Подробную инструкцию дам в главе 19, когда дойдём до PHP, — чтобы не ставить впрок то, чем ещё не пользуемся.

Заведите порядок сразу, потом будете благодарны.

Создайте папку для всех своих проектов в домашней директории:

```
projects/  
  moi-magazin/      ← наш будущий магазин  
  eksperimenty/     ← пробы и опыты
```

Два правила, которые избавят от боли:

Никаких русских букв и пробелов в названиях файлов и папок. Не `Мой магазин/каталог товаров.php`, а `moi-magazin/catalog.php`. Веб-серверы, командная строка и системы контроля версий с русскими буквами и пробелами дружат плохо, и вы поймаете «мистическую» ошибку на ровном месте.

Не храните проект в облачных папках. Не кладите код в Dropbox, iCloud или «Рабочий стол», который синхронизируется. Облако начнёт синхронизировать тысячи мелких файлов и создавать конфликтные копии. Для истории изменений есть Git — им и займёмся в главе 53.

«Я боюсь что-то сломать в терминале». Команды из этой книги ничего не ломают. Опасны ровно две вещи: `rm -rf` (удаляет безвозвратно) и всё, что начинается с `sudo` (выполняется от имени администратора). Мы их не используем. Правило простое: **не вставляйте в терминал команду, которую не понимаете**, особенно из случайных статей в интернете.

Файл сохранён? Точно? Классика жанра: правите код, обновляете браузер, ничего не меняется. В VS Code несохранённый файл помечен **точкой** рядом с именем вкладки. Включите автосохранение и забудьте про эту проблему.

Расширение файла имеет значение. `index.html` — страница. `index.txt` — просто текст, браузер покажет исходник. На Windows расширения по умолчанию скрыты, и легко создать `index.html.txt`, не заметив этого. Включите показ расширений: Проводник → Вид → «Расширения имён файлов».

Кириллица в коде. В самом тексте страницы русский, конечно, можно и нужно. Нельзя — в именах файлов, папок и названиях переменных.

Задача 3.1. Установите VS Code и сделайте три настройки из главы: перенос строк, размер шрифта, автосохранение.

Задача 3.2. Откройте терминал и выполните цепочку. Запишите, что вывела каждая команда:

```
pwd
mkdir projects
cd projects
mkdir moi-magazin
cd moi-magazin
pwd
```

Задача 3.3. Найдите на своём компьютере домашнюю папку через `pwd`, а потом откройте её же в обычном проводнике. Убедитесь, что это одно и то же место. Важно почувствовать: терминал и проводник — два окна в один и тот же дом.

Задача 3.4. Откройте любой сайт, нажмите F12 → вкладка **Elements**. Найдите заголовок страницы, дважды кликните по тексту и замените его на своё имя. Нажмите Enter.

Что произошло на самом деле — вы взломали сайт? Ответ обязательно посмотрите в приложении, он важный.

Задача 3.5. Откройте F12 → **Console** на трёх разных сайтах. Найдите хотя бы один с красными ошибками. Их полно даже у крупных компаний — это нормально и полезно знать.

Задача 3.6. Создайте на компьютере папку `projects/moi-magazin` — там мы будем строить магазин всю оставшуюся книгу.

Ответы — в Приложении Г.

В БОЮ

Откройте в проводнике корень этого репозитория и посмотрите на имена файлов:

`seller_shop.php` , `product_edit.php` , `AutoEuroPriceProvider.php` .

Заметьте: **ни одного русского слова, ни одного пробела**. Только латиница, и слова разделены подчёркиванием. Это не эстетство — это то самое правило, которое избавляет от необъяснимых сбоев на боевом сервере.

ИТОГ

Итог

- Нужны три инструмента: **редактор** (VS Code), **терминал**, **браузер** (Chrome). Все бесплатные.
- Код — это **чистый текст**. Word и «Блокнот» не годятся.
- Терминал — управление компьютером словами. Он всегда стоит в какой-то папке.
- Хватит **семи команд**: `pwd`, `ls`, `cd`, `cd ..`, `mkdir`, `touch`, `clear`.
- **Tab** дописывает, **стрелка вверх** повторяет, **Ctrl+C** прерывает.
- **F12** открывает панель разработчика: Elements, Console, Network.
- В именах файлов — **только латиница, без пробелов**.
- Не держите проект в облачной папке.

В следующей главе делаем первую страницу и открываем её в браузере. Наконец-то руками.

Первый файл на экране

ЗАЧЕМ ЭТО**Зачем эта глава**

Три главы мы читали. Хватит.

Сейчас вы сделаете страницу, откроете её в браузере и увидите на экране то, что написали своими руками. Это займёт минут пятнадцать.

Момент, когда браузер впервые показывает **вашу** страницу, — важный. Не потому, что страница получится красивой (она получится страшной), а потому, что с этой секунды интернет перестаёт быть чем-то, что делают «где-то там другие люди».

Единственная просьба: **набирайте руками**. Не копируйте мышкой. Пальцы запоминают синтаксис быстрее, чем глаза.

РАЗБИРАЕМСЯ**Делаем страницу****► Шаг 1. Заводим папку**

Откройте терминал и выполните (или сделайте то же самое мышкой в проводнике):

```
cd ~  
mkdir -p projects/moi-magazin  
cd projects/moi-magazin
```

Мы в папке будущего магазина. Отсюда и начнём.

► Шаг 2. Открываем папку в редакторе

Запустите VS Code, выберите *Файл* → *Открыть папку* и укажите `projects/moi-magazin`.

Слева появится пустое дерево файлов. Это ваш проект.

Почему открывать именно папку, а не файл: редактор тогда видит весь проект целиком — может искать по всем файлам, подсказывать пути, показывать структуру. Привыкайте работать папками.

► Шаг 3. Создаём файл

В панели слева нажмите иконку «новый файл» и назовите его:

```
index.html
```

Имя важное, не придумывайте своё. `index.html` — это имя по умолчанию. Когда браузер приходит по адресу сайта, а конкретная страница не указана, сервер отдаёт именно `index.html`. Как «главная дверь» дома.

► Шаг 4. Пишем

Наберите это. Именно наберите:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Моя первая страница</title>
</head>
<body>
  <h1>Привет! Это моя первая страница.</h1>
  <p>Я сделал её сам, и она работает.</p>
</body>
</html>
```

Сохраните: Ctrl+S (на Mac Cmd+S).

► Шаг 5. Открываем в браузере

Найдите файл в проводнике и дважды щёлкните по нему. Откроется браузер.

Вот что вы увидите:

Привет! Это моя первая страница.

Я сделал её сам, и она работает.

Работает. Вы только что сделали веб-страницу.

Посмотрите на адресную строку — там будет что-то вроде

`file:///Users/firdavs/projects/moi-magazin/index.html`. Обратите внимание на `file://` — это значит «файл с моего компьютера», а не из интернета. Пока так и нужно.

РАЗБОР ПО СЛОВАМ

Разбор по словам

Теперь разберём каждую строчку. Ни одна не написана «просто потому что».

► Строка 1: `<!DOCTYPE html>`

```
<!DOCTYPE html>
```

Читается «доктайп эйч-ти-эм-эл». Это **не тег**, а объявление: «дорогой браузер, дальше идёт современный HTML, показывай по современным правилам».

Зачем нужно: если его не написать, браузер включит «режим совместимости» — притворится браузером из 1999 года. Вёрстка поедет, и вы будете долго искать причину.

Пишется **всегда первой строкой**, ровно так, один раз на страницу. Больше о нём думать не нужно.

► Строки 2: `<html lang="ru">`

```
<html lang="ru">
```

ЧАСТЬ

ЧТО ЭТО

```
<html>
```

Корневой элемент. Внутри него живёт вся страница. Открывается в начале, закрывается в самом конце

```
lang="ru"
```

Атрибут. Сообщает: содержимое на русском

Вот и новое понятие — **атрибут**.

Атрибут — это уточнение к тегу. Пишется внутри уголков, после имени тега, в формате `имя="значение"`.

Аналогия: тег — это существительное, атрибут — прилагательное. `<html>` — «страница», `lang="ru"` — «русскоязычная».

Зачем `lang` на практике: браузер поймёт, на каком языке текст, и правильно предложит перевод. Программы для незрячих прочитают текст с русским произношением, а не с английским. Поисковики покажут ваш сайт русскоязычным пользователям.

► Строки 3–6: `<head>` — служебная часть

```
<head>
  <meta charset="UTF-8">
  <title>Моя первая страница</title>
</head>
```

`<head>` («хед», голова) — это служебная информация о странице. Всё, что внутри `<head>`, на самой странице не видно.

Аналогия: это как поля на конверте — адрес, индекс, марка. Само письмо внутри, а это данные о письме.

СТРОКА	ЧТО ДЕЛАЕТ
<code><meta charset="UTF-8"></code>	Кодировка. Объясняет браузеру, как понимать буквы
<code><title></code>	Название страницы. Оно видно на вкладке браузера и в результатах поиска

Про `charset="UTF-8"` — запомните намертво.

Компьютер хранит не буквы, а числа. Кодировка — это таблица «какое число какой букве соответствует». UTF-8 — таблица, в которой есть всё: русский, английский, таджикский, китайский, эмодзи.

Если эту строчку не написать, браузер начнёт гадать. И русский текст превратится в `ЉŹŃЄĐ,Đ²ĐµŃ,` — знаменитые «кракозябры».

Правило: `<meta charset="UTF-8">` пишется всегда, в каждом файле, без исключений.

Заметьте ещё: у `<meta>` нет закрывающего тега. Такие теги называются **одинокими** — им нечего в себе содержать, они сами по себе законченная команда. Одиноких тегов немного: `<meta>`, `<img>`, `<br>`, `<input>`, `<hr>`.

► Строки 7–10: `<body>` — то, что видно

```
<body>
  <h1>Привет! Это моя первая страница.</h1>
  <p>Я сделал её сам, и она работает.</p>
</body>
```

`<body>` («боди», тело) — всё видимое содержимое страницы. Текст, картинки, кнопки, меню — всё живёт здесь.

Правило простое: не видно на экране → это в `<head>`. Видно → это в `<body>`.

`<h1>` и `<p>` вы уже встречали в первой главе: заголовок и абзац.

► Общая схема любой HTML-страницы

<code><!DOCTYPE html></code>	← «дальше современный HTML»
<code><html></code>	← вся страница
<code><head></code>	← служебное, невидимое
кодировка	
название вкладки	
<code></head></code>	
<code><body></code>	← видимое содержимое
заголовки	
текст	
картинки	
<code></body></code>	
<code></html></code>	

Эта структура **одинакова у всех страниц в интернете**. Откройте код любого сайта — увидите её же. Через неделю вы будете набирать её не задумываясь.

► Про отступы

Заметили, что строки внутри `<head>` сдвинуты вправо? Это **отступы**, и они делаются специально.

Браузеру они не нужны — ему всё равно. Отступы нужны **человеку**: по ним сразу видно, что внутри чего лежит.

Правило: каждый вложенный уровень сдвигается вправо (обычно на 4 пробела или Tab).

Код без отступов — как текст без абзацев и знаков препинания: прочитать можно, но мучительно.

РАЗБИРАЕМСЯ

Делаем страницу похожей на магазин

Первая страница работает. Теперь наполним её чем-то осмысленным.

Замените содержимое `<body>` на это:

```
<body>
  <h1>Автозапчасти Firdavs</h1>
  <p>Запчасти для японских и немецких автомобилей. Худжанд, доставка по Таджики

  <h2>Тормозные колодки Bosch</h2>
  <p>Артикул: 0986424815</p>
  <p>Цена: 250 сомони</p>
  <p>В наличии: 7 штук</p>
  <button>Купить</button>

  <h2>Масляный фильтр Mann</h2>
  <p>Артикул: W71280</p>
  <p>Цена: 45 сомони</p>
  <p>В наличии: 23 штуки</p>
  <button>Купить</button>
</body>
```

Не забудьте поменять и `<title>` — пусть будет «Автозапчасти — мой магазин».

Сохраните и обновите страницу в браузере клавишей F5.

Автозапчасти Firdavs

Запчасти для японских и немецких автомобилей. Худжанд, доставка по Таджикистану.

Тормозные колодки Bosch

Артикул: 0986424815

Цена: 250 сомони

В наличии: 7 штук

Купить

Масляный фильтр Mann

Артикул: W71280

Цена: 45 сомони

В наличии: 23 штуки

Купить

Вот честный результат — ровно то, что вы увидите у себя.

Давайте признаем прямо: **выглядит ужасно**. Чёрный текст на белом, всё в одну колонку, кнопки серые и системные. Ни один покупатель на таком сайте ничего не купит.

И это абсолютно правильный результат. Потому что мы написали только HTML — а HTML отвечает за содержание, не за красоту. Помните первую главу? Скелет.

Смотрите, что при этом уже **работает**:

- заголовки крупнее обычного текста, а `<h1>` крупнее `<h2>` ;
- абзацы отделены друг от друга воздухом;
- кнопки нажимаются — можете щёлкнуть, они «продавливаются»;
- на вкладке браузера видно название из `<title>` ;
- русский текст читается, кракозябр нет — спасибо `charset="UTF-8"` .

Всё это браузер сделал сам, по умолчанию, потому что вы **правильно назвали части**. Сказали «это заголовок» — он и оформлен как заголовок.

Красоту наведём в третьей части, начиная с главы 9. За один вечер эта же страница превратится в нормальный магазин. Но фундамент — вот он, уже стоит.

Открылся текст кода вместо страницы. Файл сохранился как `index.html.txt`. На Windows расширения по умолчанию скрыты. Лечение: Проводник → Вид → поставить галочку «Расширения имён файлов», потом переименовать.

Вместо русского — `ÐĳŃЄĐ,Đ²ĐρŃ`. Забыли `<meta charset="UTF-8">` или сохранили файл не в UTF-8. VS Code по умолчанию сохраняет правильно, так что почти всегда дело в забытой строчке.

Поменял файл, а в браузере всё по-старому. Сначала проверьте, что файл сохранён (в VS Code у несохранённого файла точка на вкладке). Потом нажмите **Ctrl+F5** — обновление мимо кэша.

Забыл закрыть тег. Написали `<h1>` и не написали `</h1>` — дальше вся страница станет огромным заголовком. Браузер не ругается, он просто делает как понял. VS Code обычно закрывает теги сам, но проверять полезно.

Кавычки «не те». Атрибуты пишутся в прямых кавычках: `lang="ru"`. Если скопировать код из Word или мессенджера, кавычки могут превратиться в «ёлочки» — и атрибут сломается. Ещё один довод набирать руками.

Задача 4.1. Сделайте страницу из главы у себя. Убедитесь, что она открывается и русский текст читается.

Задача 4.2. Поменяйте `<title>` на своё имя. Сохраните, обновите браузер. Где именно изменился текст? (Подсказка: смотрите не на страницу.)

Задача 4.3. Добавьте в каталог третий товар — на ваш выбор, с названием, артикулом, ценой и наличием. Скопируйте структуру существующих.

Задача 4.4. Специально сломайте страницу и посмотрите, что будет:

1. Уберите `</h1>` у первого заголовка. Обновите. Что стало со страницей?
2. Верните обратно.
3. Уберите строку `<meta charset="UTF-8">`. Обновите. Что стало с русским текстом?
4. Верните обратно.

Ломать специально — лучший способ понять, зачем нужна каждая строчка. Так вы потом узнаете симптом мгновенно.

Задача 4.5. Попробуйте `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>` подряд с одинаковым текстом. Как меняется размер?

Задача 4.6. Откройте свою страницу и нажмите **F12** → **Elements**. Найдите там `<h1>`, который написали. Разверните дерево. Узнаёте свою структуру?

Задача 4.7. Откройте на любом крупном сайте код страницы (правая кнопка → «Посмотреть код страницы») и найдите в нём `<!DOCTYPE html>`, `<head>`, `<body>` и `<meta charset`. Убедитесь: у всех одно и то же.

Ответы — в Приложении Г.

В БОЮ

Откройте в этом репозитории файл `index.php` — главную страницу настоящего магазина.

Кода там много и он пока непонятен. Но найдите глазами `<head>`, `<meta charset`, `<title>` и `<body>`. Они там есть, ровно в том же порядке, что и в вашем файле.

Разница между вашей страницей и боевой — не в структуре. Структура одна. Разница в количестве наполнения. А его мы будем добавлять шестьдесят глав.

- Страница называется `index.html` — это имя главной по умолчанию.
- `<!DOCTYPE html>` — первая строка всегда, включает современный режим.
- `<html lang="ru">` — вся страница, язык русский.
- `<head>` — служебное и невидимое: кодировка, название вкладки.
- `<body>` — всё, что видно человеку.
- `<meta charset="UTF-8">` обязателен, иначе русский превратится в кракозябры.
- Атрибут — уточнение к тегу: `имя="значение"`.
- Одиночные теги (`<meta>`, `<img>`, `<br>`) не закрываются.
- Отступы нужны человеку, а не браузеру. Делайте их всегда.
- Страница без CSS выглядит ужасно — и это нормально. HTML отвечает за смысл.

Со следующей главы начинается настоящий HTML: разберём теги по-серьёзному и научимся размечать что угодно.



Продолжение следует.